

INSTITUTO INFNET
ENGENHARIA DA COMPUTAÇÃO



RELATÓRIO TÉCNICO FINAL
SISTEMAS DIGITAIS EMBARCADOS

**Integração ARM64 + FPGA com processamento MAC, telemetria e saída
HDMI**

Aluno: João Paulo Fonseca de Araújo

E-mail: joao.paraujo@al.infnet.edu.br

Projeto de Bloco: Sistemas Digitais Embarcados

[26E2_5]

Rio de Janeiro
2026

RESUMO

Este relatório documenta a consolidação do projeto desenvolvido ao longo dos Testes de Performance TP1 a TP5 da disciplina de Sistemas Digitais Embarcados. A solução final integra um programa Assembly AArch64 executado em QEMU a uma arquitetura em Verilog destinada à FPGA Tang Nano 4K. O software ARM64 gera pacotes estruturados de dez bytes contendo versão, sequência, comando, payload e CRC-8. Esses pacotes são utilizados em uma co-simulação que reproduz uma UART bit a bit, valida o protocolo, detecta CRC inválido e sequência duplicada e aciona o mesmo executor de comandos utilizado pelo protótipo físico. No lado FPGA, a arquitetura inclui sincronização e debounce dos botões, execução de comandos, unidade multiplicador-acumulador (MAC) em ponto fixo Q8.8, memória BSRAM de telemetria, indicação de status no LED onboard e geração de vídeo HDMI. A implementação física foi sintetizada e roteada no Gowin EDA para o dispositivo GW1NSR-LV4CQN48PC6/I5, com inferência de DSP MULTADDALU18X18, BSRAM, PLLVR, CLKDIV, OSER10 e buffers diferenciais ELVDS. A validação final reuniu testes automatizados, waveforms no GTKWave, relatórios de síntese e timing, programação em SRAM e observação física dos quatro modos de vídeo 00, 01, 10 e 11, incluindo o gráfico de telemetria. O resultado demonstra a integração coerente entre software de controle, protocolo de comunicação, processamento digital em FPGA e interface visual, respeitando a limitação de que o enlace ARM64-FPGA não foi implementado eletricamente em hardware e foi validado por co-simulação.

Palavras-chave: ARM64; FPGA; Tang Nano 4K; Verilog; QEMU; MAC; BSRAM; HDMI; UART; CRC; telemetria.

SUMÁRIO

1 INTRODUÇÃO	4
2 OBJETIVOS	5
3 EVOLUÇÃO DO PROJETO: TP1 AO TP5	6
4 ARQUITETURA FINAL DO SISTEMA	7
5 IMPLEMENTAÇÃO ARM64 E GERAÇÃO DOS PACOTES	8
6 CO-SIMULAÇÃO E VALIDAÇÃO DO PROTOCOLO	9
7 IMPLEMENTAÇÃO DO DATAPATH FPGA	12
8 SÍNTESE, PLACE & ROUTE E TIMING	14
9 VALIDAÇÃO FÍSICA NA TANG NANO 4K	19
10 RESULTADOS E DISCUSSÃO	22
11 CONCLUSÃO	23
REFERÊNCIAS	24

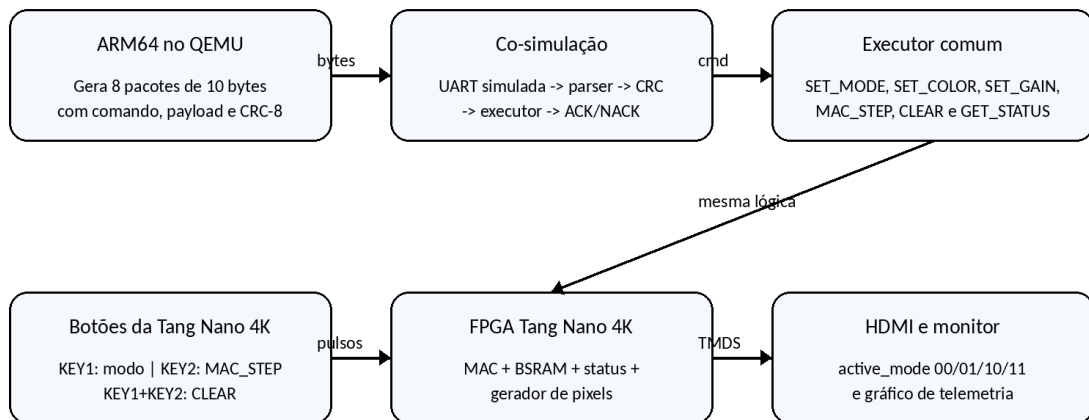
1 INTRODUÇÃO

O projeto final de Sistemas Digitais Embarcados representa a consolidação de uma sequência de protótipos desenvolvidos ao longo de cinco TPs. A proposta inicial partiu de um controlador de vídeo no qual um processador ARM produziria dados de pixel e uma FPGA cuidaria da temporização e da exibição. Ao longo das entregas, as limitações de memória da Tang Nano 4K, a indisponibilidade do Raspberry Pi Zero 2W físico e a necessidade de demonstrar processamento, comunicação e observabilidade conduziram a uma arquitetura final diferente da concepção inicial, porém preservando o objetivo central de cooperação entre software ARM64 e lógica reconfigurável.

Na solução final, o lado ARM64 é executado no QEMU e gera comandos estruturados. Esses bytes alimentam uma co-simulação Verilog na qual uma UART simulada transporta o pacote, um parser valida os campos e o CRC, o executor aplica comandos e uma resposta ACK ou NACK é produzida. Em paralelo, a implementação física na Tang Nano 4K utiliza os dois botões onboard como fonte local de comandos equivalentes, de modo que MAC, telemetria, LED e HDMI possam ser demonstrados sem afirmar a existência de um enlace serial físico que não foi montado.

A saída HDMI é utilizada como interface de observação do estado interno. O valor `active_mode` possui quatro combinações: 00 exibe barras de cores; 01 exibe um padrão xadrez; 10 exibe um gradiente RGB; e 11 apresenta a telemetria da unidade MAC. O LED onboard foi reservado para status de atividade e erro. Assim, a imagem exibida não constitui apenas um efeito visual: ela torna observável o estado de uma arquitetura digital composta por entrada, controle, processamento, memória e saída.

Arquitetura final do projeto ARM64 + FPGA



Integração ARM64-FPGA: validada por co-simulação Hardware: validado fisicamente na Tang Nano 4K

Figura 1 - Arquitetura conceitual da solução final.

Fonte: elaboração própria a partir das evidências do projeto (2026).

2 OBJETIVOS

2.1 Objetivo geral

Consolidar os cinco TPs em um protótipo integrado capaz de demonstrar, de forma reproduzível, a interação conceitual entre software AArch64 e lógica FPGA, incluindo protocolo de comandos com detecção de erros, processamento MAC em ponto fixo, armazenamento de telemetria e visualização por HDMI na Tang Nano 4K.

2.2 Objetivos específicos

- Gerar, em Assembly AArch64 executado no QEMU, uma sequência determinística de oito pacotes de comando.
- Validar os pacotes por CRC-8 e disponibilizá-los à co-simulação em formato binário e hexadecimal.
- Receber bytes por uma UART simulada, detectar CRC inválido, comando inválido, tamanho inválido e sequência duplicada.
- Executar comandos de controle e processamento por meio de um executor comum às validações lógica e física.
- Implementar uma unidade MAC 16 x 16 bits com acumulador de 32 bits, resultado Q8.8 e indicação de overflow.
- Armazenar resultados recentes em BSRAM para formar uma base de telemetria.

- Gerar vídeo HDMI estável e associar os quatro valores de `active_mode` a padrões visuais distintos.
- Utilizar o LED onboard para sinalizar comando aceito, erro de protocolo e erro grave.
- Confirmar síntese, place and route, recursos dedicados, pinagem, timing e programação física da Tang Nano 4K.

3 EVOLUÇÃO DO PROJETO: TP1 AO TP5

3.1 TP1 - arquitetura inicial ARM-FPGA-VGA

O TP1 definiu a arquitetura preliminar de um controlador VGA com framebuffer ARM→FPGA. O Raspberry Pi Zero 2W seria responsável por calcular conteúdo de tela e enviá-lo por SPI, enquanto a Tang Nano 4K armazenaria os pixels em BRAM e geraria sincronismos VGA em 640 x 480 @ 60 Hz. Nessa etapa foram introduzidos os conceitos de endereço de pixel, framebuffer, geração de cores e divisão de responsabilidades entre CPU e FPGA.

3.2 TP2 - sincronismo de vídeo, padrões e fluxo de build

No TP2, a arquitetura foi concretizada por módulos Verilog de sincronismo e geração de padrões, testbench com Icarus Verilog e inspeção de waveforms no GTKWave. O lado ARM evoluiu para rotinas com loops, condicionais e escrita em buffer. A comunicação SPI ainda era tratada como etapa planejada ou emulada, e o uso de VGA permaneceu como referência de temporização de vídeo.

3.3 TP3 - máquina de estados, entradas físicas e QEMU

O TP3 introduziu uma FSM receptora de comandos, sincronizador de dois flip-flops, debounce de aproximadamente 20 ms e reset de inicialização. A indisponibilidade do Raspberry Pi Zero 2W físico levou a frente ARM64 a ser validada por QEMU, GDB e objdump. Também foram estudados GPIO mapeados em memória, loops, condicionais, tabela de salto e parser de comandos. Essa etapa estabeleceu a separação que seria mantida até o projeto final: FPGA validada fisicamente e ARM64 validado por emulação.

3.4 TP4 - MAC, BRAM, DSP e telemetria

No TP4, o sistema ganhou processamento numérico efetivo. Foi implementada uma unidade MAC com multiplicador 16 x 16 bits e acumulador saturado de 32 bits, buscando a inferência de DSP dedicado. Memórias em bloco foram utilizadas para coeficientes e amostras, e o relatório de síntese confirmou o uso de recursos DSP e BSRAM. No lado ARM

foram acrescentadas aritmética multi-palavra, conversões numéricas, lookup tables, operações bitwise e NEON SIMD. Também surgiu um mecanismo de comunicação simulada com checksum e registro de telemetria.

3.5 TP5 - protocolo, biblioteca Assembly e integração

O TP5 consolidou o protótipo em torno de uma biblioteca Assembly modular e de um protocolo com envio, recepção, checksum, end-of-frame, handshaking e telemetria. O MAC foi revisado com casos de borda adicionais, e a arquitetura FPGA continuou sendo validada na Tang Nano 4K. O enlace elétrico ARM-FPGA permaneceu pendente, mas a lógica de protocolo e o processamento de ambos os lados estavam maduros para a consolidação final.

3.6 Consolidação no projeto final

O projeto final substituiu o framebuffer completo por geração procedural de pixels e migrou a demonstração física para HDMI, utilizando a base oficial da Sipeed. O protocolo foi formalizado em pacotes de dez bytes com CRC-8; os bytes produzidos pelo QEMU passaram a alimentar diretamente um testbench que simula a UART; e a Tang Nano passou a demonstrar localmente o mesmo executor por meio dos botões. Essa decisão fecha o ciclo dos TPs sem mascarar a ausência do canal físico ARM-FPGA.

4 ARQUITETURA FINAL DO SISTEMA

4.1 Divisão das responsabilidades

Subsistema	Responsabilidade na solução final
ARM64/QEMU	Montar os oito comandos, calcular CRC-8 e escrever 80 bytes no fluxo de saída.
Bridge Python	Converter commands.bin para commands.hex e validar a estrutura dos pacotes.
Co-simulação Verilog	Reproduzir o transporte UART, parser, validação, execução e respostas.
Tang Nano 4K	Executar controle, MAC, BSRAM, status, geração procedural de pixels e HDMI.
Botões onboard	Gerar comandos locais equivalentes no protótipo físico.
LED onboard	Indicar atividade, erro de protocolo/comando e erro grave.
Monitor HDMI	Exibir active_mode e a telemetria do MAC.

4.2 Protocolo de comando

Cada comando possui dez bytes. O primeiro byte 0xA5 marca o início do pacote. Em seguida são enviados versão, número de sequência, identificador do comando, tamanho do

payload, quatro bytes de payload e um CRC-8 com polinômio 0x07 calculado sobre os bytes 1 a 8. O número de sequência permite rejeitar retransmissões acidentais de um comando já processado.

Byte	Campo	Função
0	SOF = 0xA5	Marca o início do pacote.
1	Versão = 0x01	Identifica a versão do protocolo.
2	Sequência	Distingue comandos e permite detectar duplicatas.
3	Comando	SET_MODE, SET_COLOR, SET_GAIN, MAC_STEP, CLEAR, GET_STATUS ou PING.
4	Length	Quantidade de bytes úteis no payload, de 0 a 4.
5-8	Payload	Parâmetros do comando; bytes não usados permanecem em zero.
9	CRC-8	Detecta corrupção acidental do pacote.

4.3 Caminho físico e caminho de co-simulação

A integração ARM64-FPGA possui dois caminhos de validação complementares. Na co-simulação, os bytes produzidos pelo QEMU entram em uma UART Verilog e percorrem o parser, o executor e o construtor de resposta. No hardware físico, os botões da placa geram comandos locais e alimentam o executor, permitindo observar a mesma lógica de controle, MAC, BSRAM, LED e HDMI. Portanto, o trabalho demonstra integração funcional entre as partes, mas não uma transmissão UART elétrica em tempo real entre QEMU e a placa.

5 IMPLEMENTAÇÃO ARM64 E GERAÇÃO DOS PACOTES

5.1 Build, execução no QEMU e verificação

O fluxo final é automatizado por Makefiles. O assembler aarch64-linux-gnu-as produz os objetos de main.s, packet.s e syscalls.s; o linker gera packet_gen; o qemu-aarch64 executa o binário e redireciona a saída para commands.bin. Em seguida, bin_to_hex.py cria commands.hex e check_packets.py valida cada pacote, incluindo SOF, versão e CRC.

```
jotape@DARKIN: /mnt/d/estu$ make clean
make -C arm64 clean
make[1]: Entering directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/arm64'
rm -rf build
make[1]: Leaving directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/arm64'
make -C fpga/sin clean
make[1]: Entering directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/fpga/sin'
rm -rf build
make[1]: Leaving directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/fpga/sin'
rm -f fpga/sin/commands.hex
jotape@DARKIN: /mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project$ make all
make -C arm64 check
make[1]: Entering directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/arm64'
mkdir -p build
aarch64-linux-gnu-gcc -g -g build/main.o src/main.s
src/main.s: Assembler messages:
src/main.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-gcc -g -g build/packet.o src/packet.s
aarch64-linux-gnu-gcc -g -g build/syscalls.o src/syscalls.s
src/syscalls.s: Assembler messages:
src/syscalls.s: Warning: end of file not at end of a line; newline inserted
aarch64-linux-gnu-ld -static -o start -g build/packet.o build/main.o build/syscalls.o
some aarch64 build/packet.o > build/commands.bin
python3 ./bridge/bin_to_hex.py build/commands.bin ../fpga/sin/commands.hex
build/commands.bin: 80 bytes -> ../fpga/sin/commands.hex
python3 ./bridge/check_packets.py build/commands.bin
packets 00 seq=00 cmd=00 len=4 crc=00 PASS
packets 01 seq=01 cmd=01 MODE len=1 crc=01 PASS
packets 02 seq=02 cmd=02 LED len=3 crc=02 PASS
packets 03 seq=03 cmd=03 GAIN len=2 crc=03 PASS
packets 04 seq=04 cmd=04 STOP len=3 crc=04 PASS
packets 05 seq=05 cmd=05 MAC len=4 crc=05 PASS
packets 06 seq=06 cmd=06 LED len=4 crc=06 PASS
packets 07 seq=07 cmd=07 STATUS len=6 crc=07 PASS
python3 ./8_PASS_0_FALHA.py
make[1]: Leaving directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/arm64'
make -C fpga/sin all
make[1]: Entering directory '/mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project/fpga/sin'
mkdir -p build
iverilog -g2012 -wall -s build/tb_mac_unit.vvp ../rtl/dsp/mac_unit.v tb_mac_unit.v
warning: Some design elements have no explicit time unit and/or
: time precision. This may cause confusing timing results.
: Affected design elements are:
: module mac_unit declared here: ../rtl/dsp/mac_unit.v:1
vvp build/tb_mac_unit.vvp
VCD info: dumpfile build/tb_mac_unit.vcd opened for output.
PASS tb_mac_unit
tb_mac_unit.v:79: $finish called at 220000 (lps)
iverilog -g2012 -wall -s build/tb_led_status.vvp ../rtl/in/led_status.v tb_led_status.v
vvp build/tb_led_status.vvp
VCD info: dumpfile build/tb_led_status.vcd opened for output.
PASS tb_led_status
tb_led_status.v:80: $finish called at 430000 (lps)
iverilog -g2012 -wall -s build/tb_pattern_stream.vvp ../rtl/video/pattern_stream.v tb_pattern_stream.v
warning: Some design elements have no explicit time unit and/or
: time precision. This may cause confusing timing results.
: Affected design elements are:
: module pattern_stream declared here: ../rtl/video/pattern_stream.v:1
vvp build/tb_pattern_stream.vvp
VCD info: dumpfile build/tb_pattern_stream.vcd opened for output.
PASS tb_pattern_stream
tb_pattern_stream.v:87: $finish called at 2620000 (lps)
iverilog -g2012 -wall -s build/tb_arm_fpga_protocol.vvp ../rtl/control/uart_rx.v ../rtl/control/packet_parser.v ../rtl/control/response_builder.v ../rtl/control/command_executor.v ../rtl/dsp/mac_unit.v ../rtl/in/led_status.v ../rtl/protocol_sys
tem.v tb_arm_fpga_protocol.v
../rtl/protocol_system.v:1: warning: timescale for protocol_system inherited from another file.
../rtl/in/led_status.v:12: ... The inherited timescale is here.
warning: Some design elements have no explicit time unit and/or
: time precision. This may cause confusing timing results.
: Affected design elements are:
: module command_executor declared here: ../rtl/control/command_executor.v:1
```

Figura 2 - Execução do fluxo ARM64 no WSL, incluindo montagem, link, QEMU e validação dos oito pacotes.
Fonte: elaboração própria a partir das evidências do projeto (2026).

```
jotape@DARKIN: /mnt/d/estu$ wc -c arm64/build/commands.bin
80 arm64/build/commands.bin
jotape@DARKIN: /mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project$ wc -l fpga/sin/commands.hex
80 fpga/sin/commands.hex
jotape@DARKIN: /mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project$ head -20 fpga/sin/commands.hex
a5
01
00
7f
04
11
22
33
44
26
a5
01
01
01
01
01
00
00
00
00
91
jotape@DARKIN: /mnt/d/estudos/infnet/blocos/ph/vga-framebuffer-arm-fpga/final_project$ |
```

Figura 3 - Verificação de commands.bin com 80 bytes, commands.hex com 80 linhas e amostra dos bytes gerados.
Fonte: elaboração própria a partir das evidências do projeto (2026).

A evidência confirma oito pacotes de dez bytes, totalizando 80 bytes. A validação terminou em 8 PASS e 0 FALHA, o que demonstra que o gerador AArch64, o CRC e a conversão para hexadecimal produziram o conjunto esperado para a simulação.

6 CO-SIMULAÇÃO E VALIDAÇÃO DO PROTOCOLO

6.1 Testes automatizados

A pasta fpga/sim possui quatro ensaios principais. tb_mac_unit valida o núcleo aritmético; tb_led_status verifica os padrões do LED; tb_pattern_stream valida a representação dos quatro valores de active_mode; e tb_arm_fpga_protocol usa os 80 bytes

gerados pelo ARM64 para fechar a cadeia arquivo -> UART simulada -> parser -> executor -> resposta.



Figura 4 - PASS do tb_mac_unit.

Fonte: elaboração própria (2026).



Figura 5 - PASS do tb_led_status.

Fonte: elaboração própria (2026).

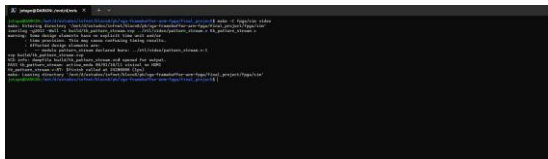


Figura 6 - PASS do tb_pattern_stream.

Fonte: elaboração própria (2026).

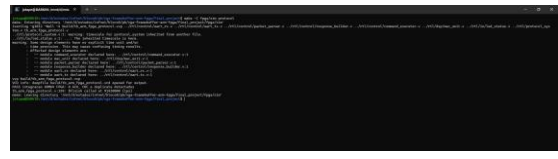


Figura 7 - PASS da integração ARM64-FPGA.

Fonte: elaboração própria (2026).

O resultado final da integração registra oito ACKs, um erro de CRC e uma sequência duplicada detectados. Essa combinação é importante porque testa tanto o caminho nominal quanto situações de falha, evitando uma validação baseada apenas em comandos corretos.

6.2 Pacote válido e ACK

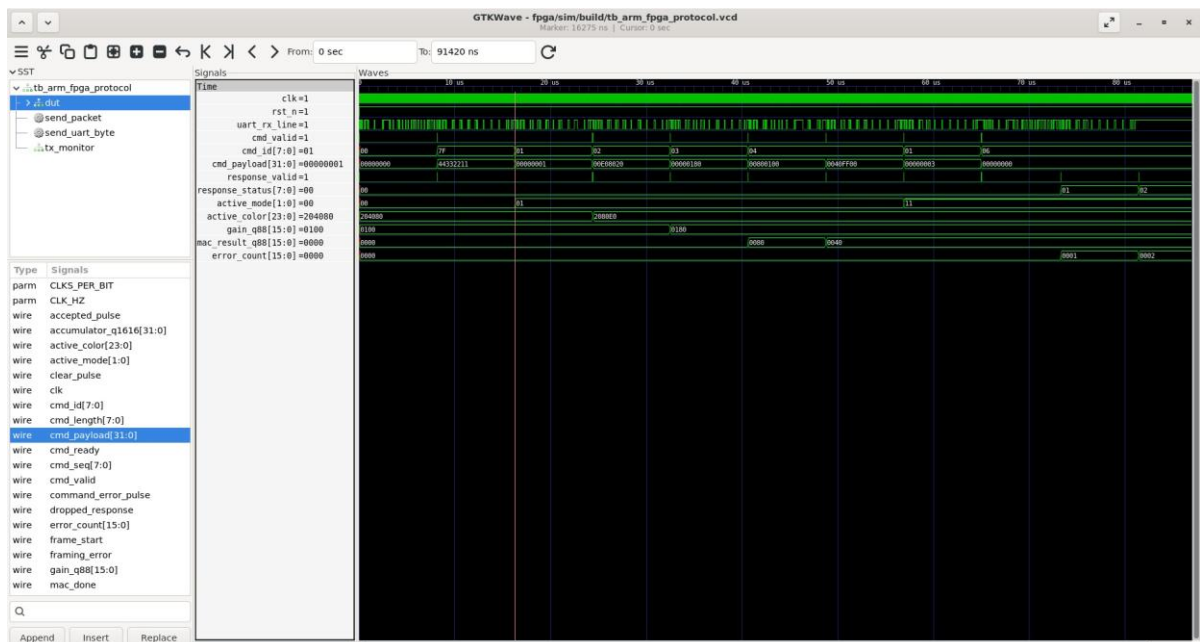


Figura 8 - Waveform da integração mostrando pacote válido, resposta ACK e atualização de estado.

Fonte: elaboração própria a partir das evidências do projeto (2026).

Na região de pacote válido, cmd_valid é gerado e a resposta utiliza status 0, correspondente a ACK. O estado de controle pode ser atualizado após a aceitação do comando, confirmando que a validação do protocolo precede a execução.

6.3 Detecção de CRC inválido

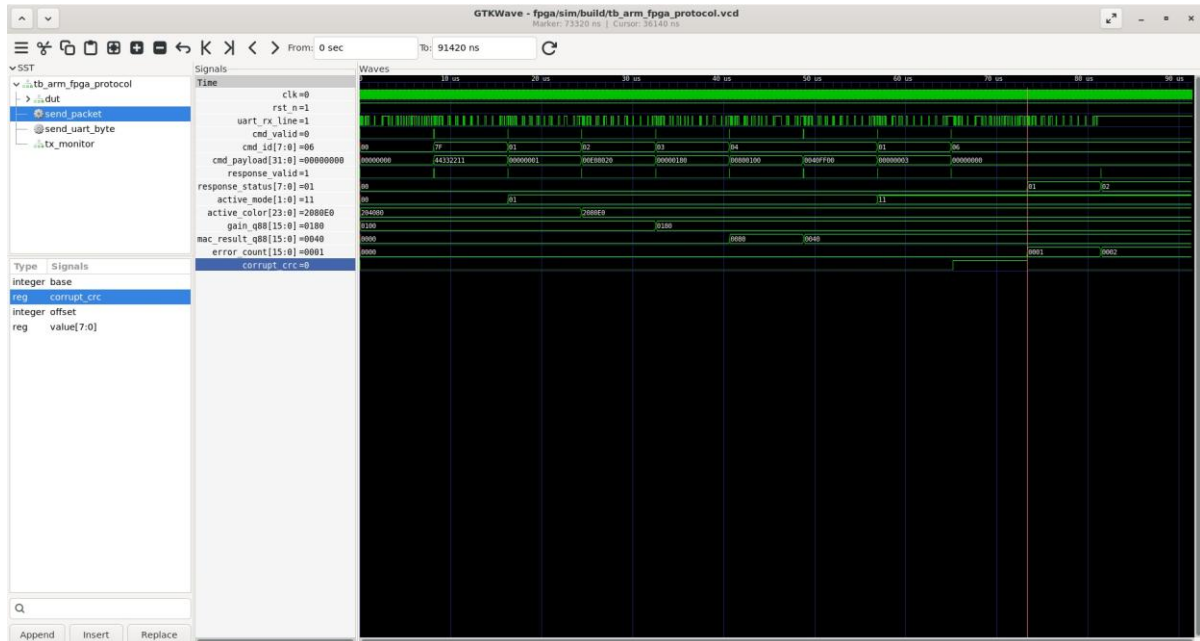


Figura 9 - Waveform do pacote com CRC propositalmente corrompido, status 01 e incremento do contador de erros.

Fonte: elaboração própria a partir das evidências do projeto (2026).

O testbench altera o último byte do pacote para produzir um CRC inválido. A resposta assume status 01 e error_count passa de 0000 para 0001. O comando não é aceito como uma transação válida, demonstrando que a integridade do pacote é verificada antes da execução.

6.4 Detecção de sequência duplicada

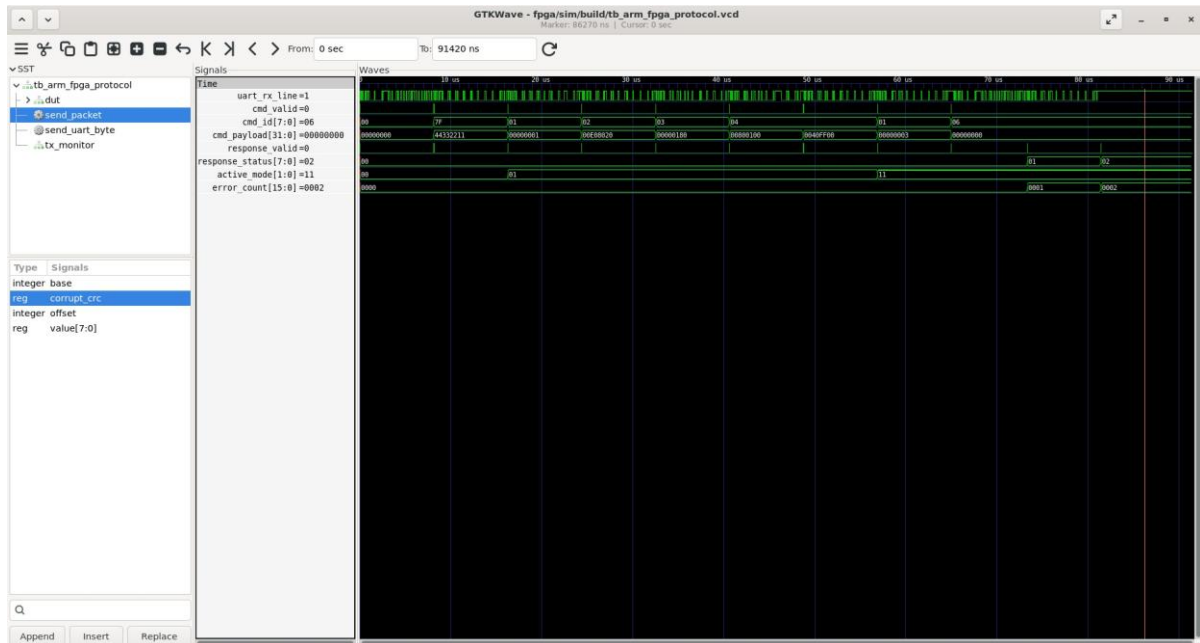


Figura 10 - Waveform de sequência duplicada, status 02 e error_count igual a 0002.

Fonte: elaboração própria a partir das evidências do projeto (2026).

Após o erro de CRC, o testbench retransmite um pacote cuja sequência já havia sido processada. O parser retorna status 02, eleva o contador total de erros para 0002 e não gera uma nova execução do comando. Esse mecanismo impede efeitos duplicados causados por retransmissões.

7 IMPLEMENTAÇÃO DO DATAPATH FPGA

7.1 Entrada física, sincronização e debounce

Os sinais dos botões são assíncronos em relação ao clock interno e também estão sujeitos ao bounce mecânico. Por isso, cada botão passa por sincronização e debounce antes de gerar pulsos de comando. KEY1 percorre os modos de vídeo; KEY2 gera MAC_STEP; e o acionamento conjunto produz CLEAR. Essa lógica permite demonstrar o datapath físico mesmo sem UART externa.

7.2 Unidade MAC em Q8.8

A unidade MAC recebe dois operandos signed de 16 bits em formato Q8.8, multiplica os valores e soma o produto a um acumulador de 32 bits. O acumulador preserva maior faixa numérica durante a soma e o resultado é disponibilizado novamente em Q8.8. O sinal overflow identifica saturação ou condição grave para a lógica de status.

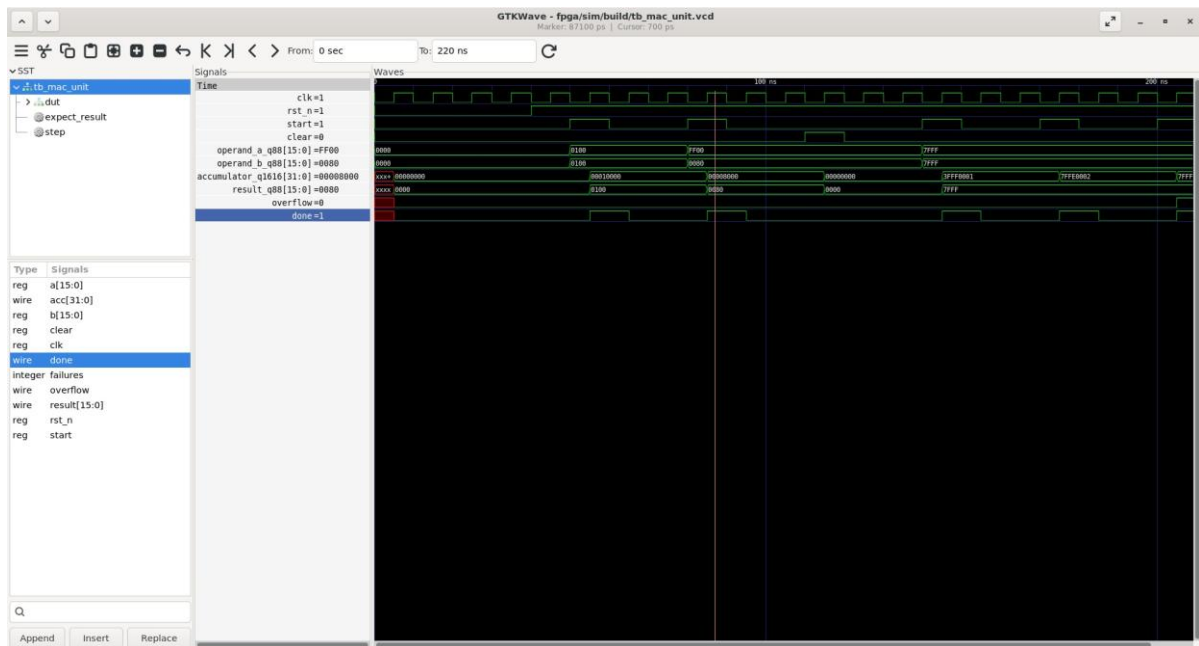


Figura 11 - Waveform da unidade MAC mostrando start, operandos, acumulador, resultado, overflow e done.

Fonte: elaboração própria a partir das evidências do projeto (2026).

A waveform evidencia operações sucessivas. No ponto destacado, os operandos FF00 (-1,0 em Q8.8) e 0080 (+0,5) produzem atualização coerente do acumulador e result_q88 igual a 0080 (+0,5), considerando o resultado acumulado da operação anterior. O pulso done marca a conclusão de cada passo.

7.3 BSRAM de telemetria

A memória de telemetria armazena resultados concluídos pelo MAC e fornece amostras ao gerador de vídeo. Em vez de reservar memória para um framebuffer completo, a solução final calcula os pixels proceduralmente e dedica a BSRAM ao histórico numérico. Essa decisão reduz drasticamente o consumo de memória e permite transformar resultados internos em um gráfico HDMI.

7.4 LED onboard como canal de status

O LED onboard foi desacoplado da representação de active_mode. Um comando aceito gera uma piscada curta; um erro de protocolo/comando gera duas piscadas; e um erro grave mantém uma piscada rápida contínua até CLEAR. A validação temporal dessa lógica é registrada no testbench.

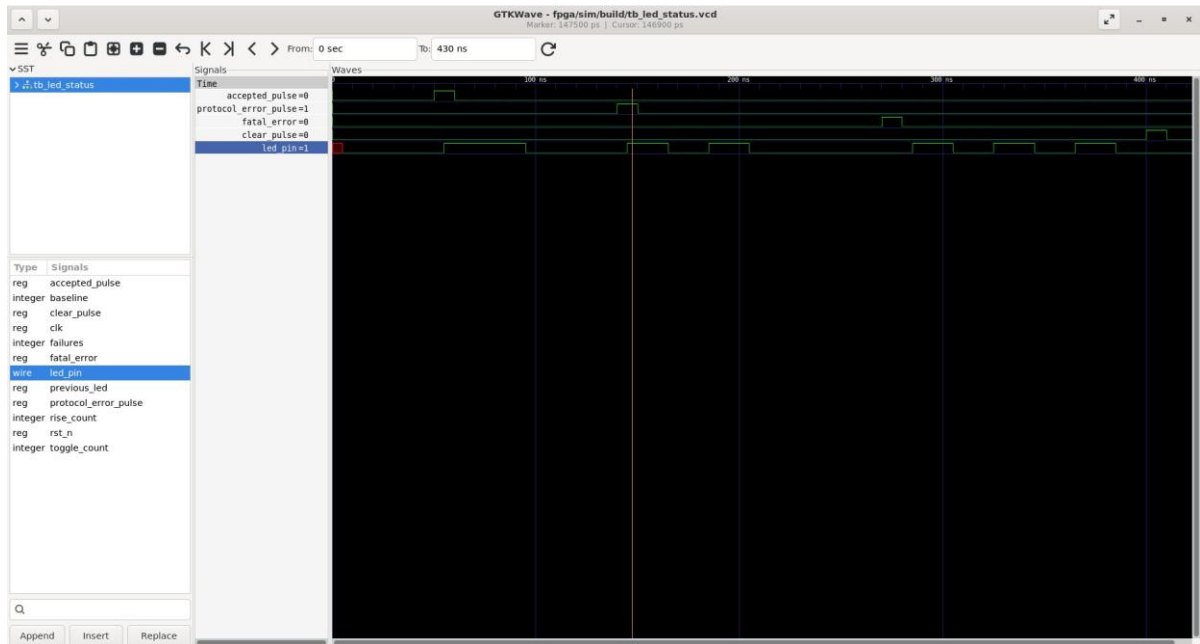


Figura 12 - Waveform de led_status com eventos de comando aceito, erro de protocolo, erro grave, clear e saída led_pin.

Fonte: elaboração própria a partir das evidências do projeto (2026).

7.5 Geração HDMI e active_mode

O pipeline de vídeo reutiliza blocos oficiais SVO/HDMI da Sipeed e insere pattern_stream como fonte dos pixels. active_mode seleciona barras, xadrez, gradiente ou telemetria. Dois pequenos indicadores no canto superior direito representam os bits mode[1] e mode[0], permitindo verificar visualmente o estado binário do sistema. A geração procedural evita a necessidade de armazenar um quadro 640 x 480 completo.

8 SÍNTESE, PLACE & ROUTE E TIMING

8.1 Projeto final no Gowin

O projeto físico foi montado sobre a base HDMI oficial, mantendo os módulos SVO, PLLVR e CLKDIV. top_final foi selecionado como top module, e as constraints finais substituíram as constraints do exemplo original. protocol_system, UART, parser e testbenches permanecem no fluxo de co-simulação e não integram o top físico atual.

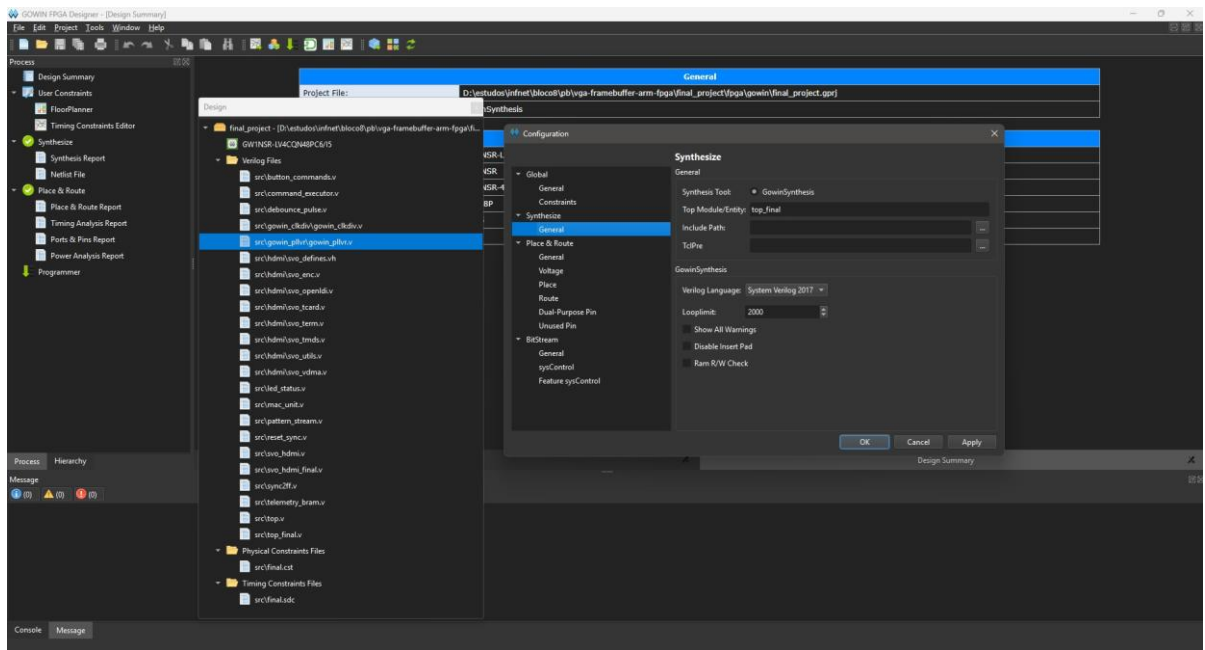


Figura 13 - Árvore do projeto Gowin e configuração de top_final como módulo principal.

Fonte: elaboração própria a partir das evidências do projeto (2026).

8.2 Síntese e utilização de recursos

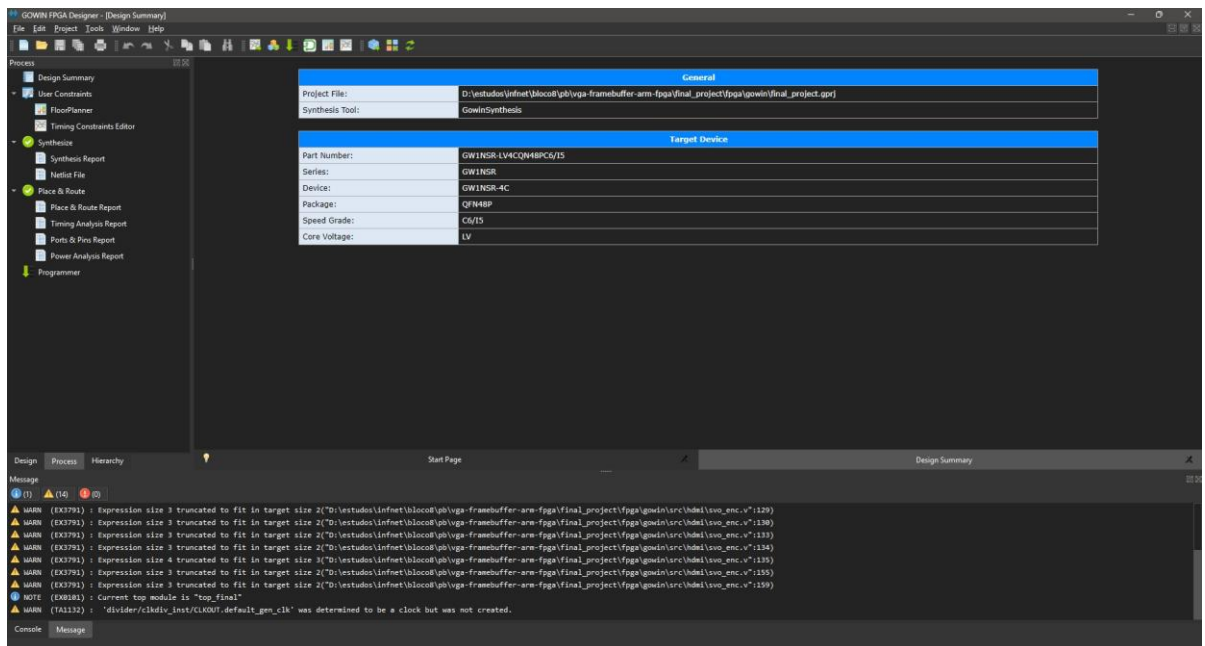


Figura 14 - Design Summary após síntese e Place & Route, sem erros e com top module top_final.

Fonte: elaboração própria a partir das evidências do projeto (2026).

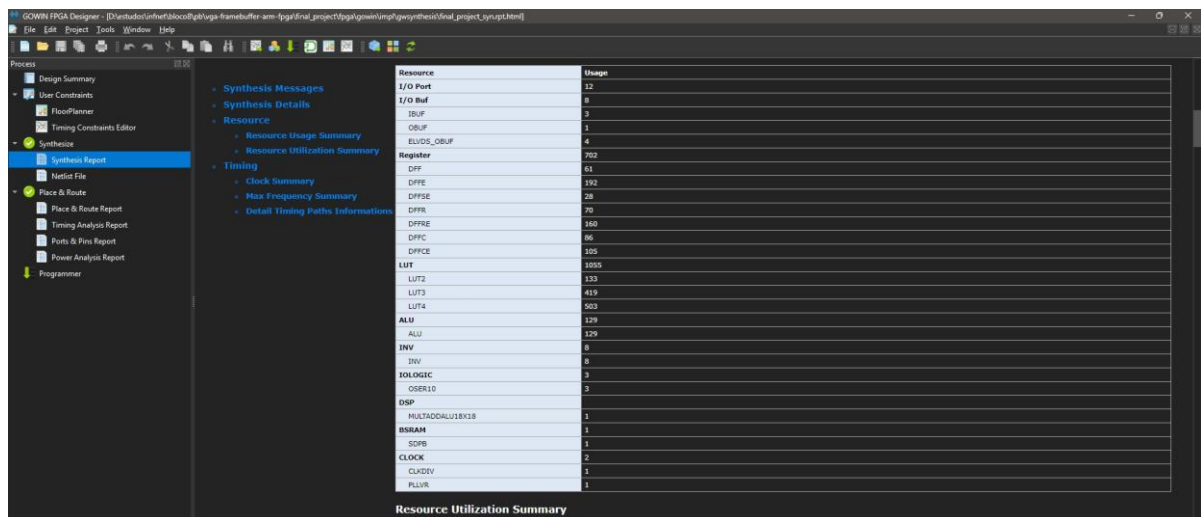


Figura 15 - Resource Utilization Summary do projeto final.

Fonte: elaboração própria a partir das evidências do projeto (2026).

A síntese confirmou recursos dedicados coerentes com a arquitetura. Foram utilizados 1 bloco DSP MULTADDALU18X18, 1 BSRAM do tipo SDPB, 1 PLLVR, 1 CLKDIV, 3 OSER10 e 4 ELVDS_OBUF, além de 1055 LUTs e 702 registradores. Esses dados comprovam que o MAC foi mapeado em hardware aritmético dedicado, a telemetria utiliza memória em bloco e o pipeline HDMI emprega os recursos de clock, serialização e saída diferencial esperados.

Recurso	Uso observado	Interpretação
MULTADDALU18X18	1	DSP dedicado utilizado pelo produto da unidade MAC.
BSRAM / SDPB	1	Memória em bloco utilizada pela telemetria.
PLLVR	1	Geração do clock rápido do pipeline HDMI.
CLKDIV	1	Derivação do clock de pixels de 25,2 MHz.
OSER10	3	Serialização dos canais de dados TMDS.
ELVDS_OBUF	4	Três pares de dados HDMI e um par de clock.
LUT	1055	Controle, comparações e lógica combinacional.
Register	702	Estados, contadores, pipelines e registradores de controle.

8.3 Análise temporal



Figura 16 - Resumo de clocks, frequência máxima e Total Negative Slack após implementação.
Fonte: elaboração própria a partir das evidências do projeto (2026).

O clock externo foi reconhecido em 27 MHz. A PLL gera 126 MHz, e o divisor produz 25,2 MHz para o domínio de pixels. Para esse domínio, o relatório apresenta restrição de 25,2 MHz e Fmax estimada em 35,252 MHz. O Total Negative Slack Summary registra TNS = 0,000 e zero endpoints negativos para os domínios listados. Em capturas detalhadas do relatório de timing, os caminhos de setup e hold observados apresentaram slack positivo, reforçando a ausência de violação crítica no conjunto analisado.

8.4 Pinagem e constraints físicas

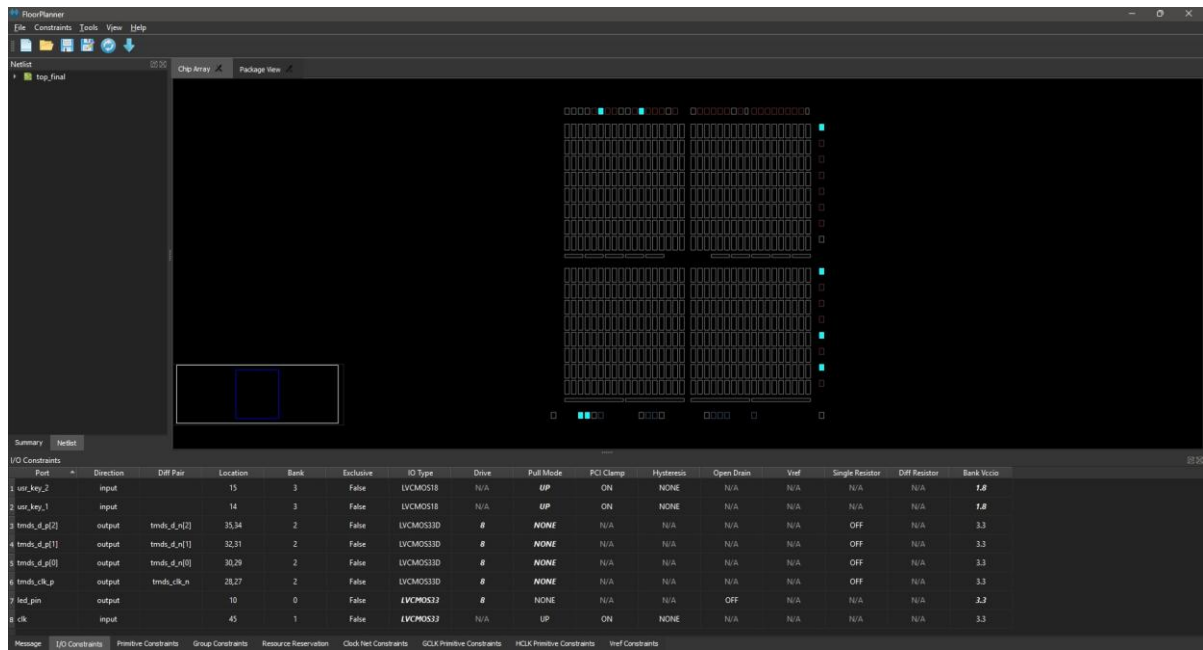


Figura 17 - FloorPlanner com pinagem final de clock, botões, LED e pares HDMI.

Fonte: elaboração própria a partir das evidências do projeto (2026).

A pinagem final associa clk ao pino 45, usr_key_1 ao pino 14, usr_key_2 ao pino 15 e led_pin ao pino 10. Os pares HDMI ocupam 30/29, 32/31, 35/34 para os três canais de dados e 28/27 para o clock diferencial. A captura também confirma bancos e níveis de I/O utilizados na implementação.

8.5 Programação em SRAM

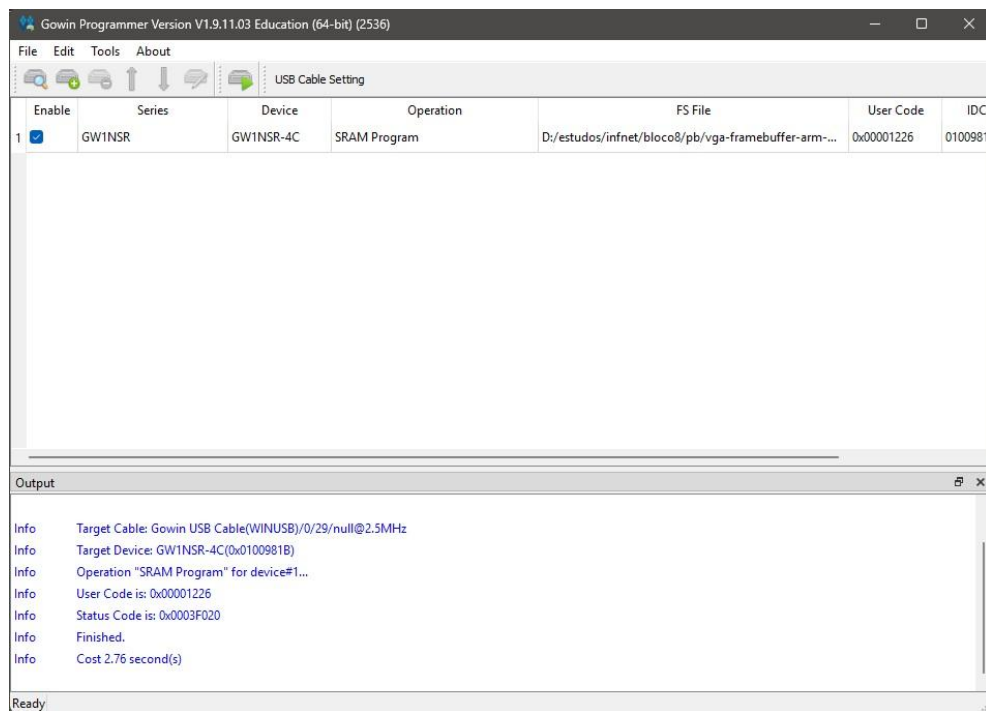


Figura 18 - Gowin Programmer após programação SRAM concluída com sucesso.

Fonte: elaboração própria a partir das evidências do projeto (2026).

O bitstream final foi carregado na Tang Nano 4K utilizando SRAM Program. A mensagem Finished confirma a conclusão da gravação temporária utilizada para a validação física.

9 VALIDAÇÃO FÍSICA NA TANG NANO 4K

9.1 active_mode 00 e 01



Figura 19 - active_mode 00: barras de cores.

Fonte: elaboração própria (2026).

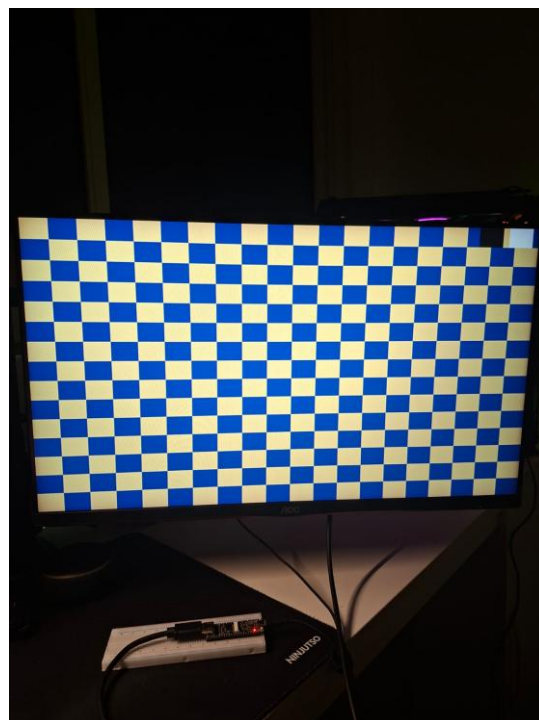


Figura 20 - active_mode 01: padrão xadrez.

Fonte: elaboração própria (2026).

As duas primeiras telas confirmam a troca de estado por comando local e a geração contínua de vídeo. No modo 00, o monitor recebe barras verticais; no modo 01, o padrão muda para xadrez. Os indicadores binários no canto superior direito acompanham o valor de active_mode.

9.2 active_mode 10

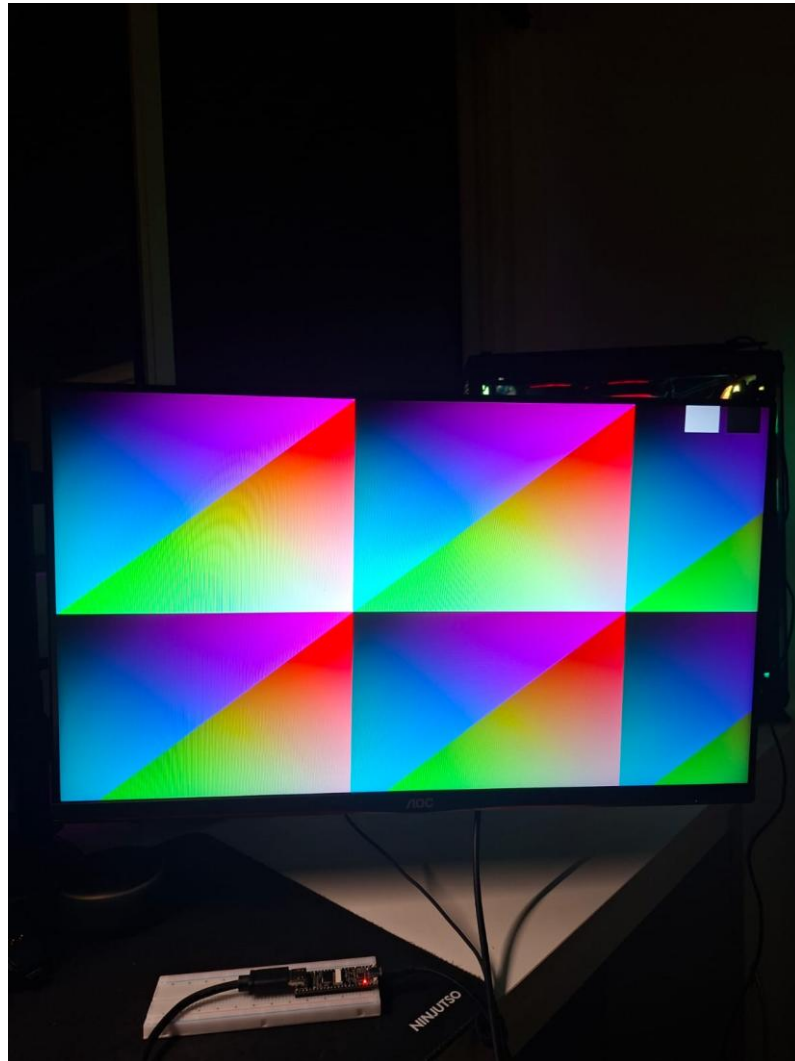


Figura 21 - active_mode 10: padrão de gradiente RGB exibido fisicamente no monitor.

Fonte: elaboração própria a partir das evidências do projeto (2026).

9.3 active_mode 11 e telemetria

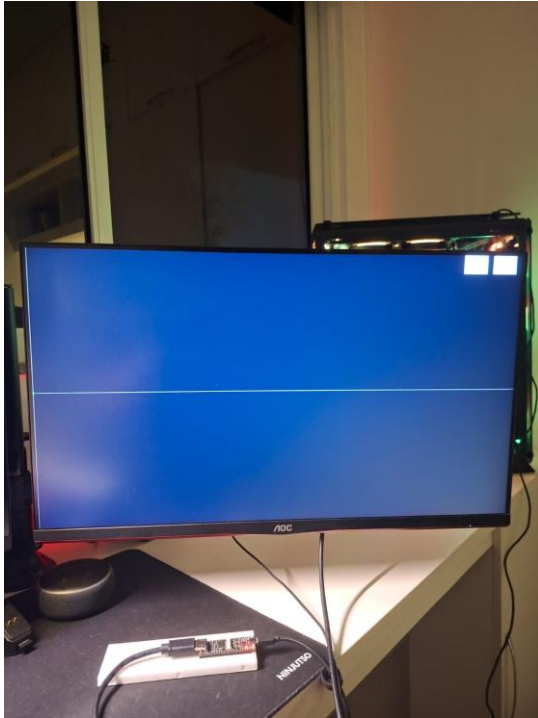


Figura 22 - active_mode 11 antes de novas amostras MAC.

Fonte: elaboração própria (2026).

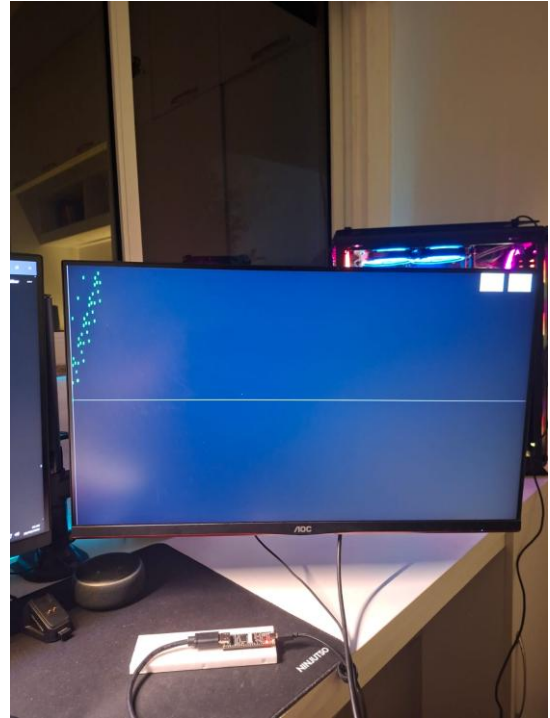


Figura 23 - active_mode 11 após MAC_STEP, com amostras de telemetria no gráfico.

Fonte: elaboração própria (2026).

O modo 11 utiliza um fundo escuro, uma linha horizontal de referência e pontos derivados das amostras armazenadas. A comparação entre as Figuras 22 e 23 mostra que, após operações MAC, novos pontos passam a ser exibidos na região do gráfico. Essa evidência fecha o caminho físico botão -> comando -> MAC -> BSRAM -> leitura de telemetria -> pattern_stream -> HDMI.

9.4 Observação sobre o LED físico

O funcionamento lógico dos padrões do LED foi comprovado pelo testbench e pela waveform apresentada na Figura 12. As evidências estáticas deste pacote não registram temporalmente uma piscada física, pois uma fotografia isolada não comprova frequência ou quantidade de pulsos. Assim, este relatório atribui a validação dos padrões de status à simulação e não extrapola a evidência para uma medição física temporal do LED.

10 RESULTADOS E DISCUSSÃO

10.1 Síntese dos resultados

Item avaliado	Resultado	Evidência principal
Assembly AArch64 no QEMU	Aprovado	8 pacotes válidos; commands.bin = 80 bytes; commands.hex = 80 linhas.
MAC	Aprovado	tb_mac_unit PASS, waveform e DSP MULTADDALU18X18 inferido.
LED de status	Aprovado em simulação	tb_led_status PASS e waveform de padrões.
Geração de vídeo	Aprovado	tb_pattern_stream PASS e quatro modos HDMI físicos.
Protocolo	Aprovado em co-simulação	8 ACKs, 1 CRC inválido e 1 duplicata detectados.
BSRAM/telemetria	Aprovado	BSRAM inferida e gráfico físico com amostras após MAC_STEP.
Síntese e P&R	Aprovado	Gowin sem erros e bitstream gerado.
Timing	Aprovado no relatório apresentado	25,2 MHz requeridos, Fmax 35,252 MHz e TNS 0,000.
Programação física	Aprovado	SRAM Program finalizado com sucesso.
Enlace ARM64-FPGA físico	Não implementado	Integração realizada por co-simulação; comandos físicos entram pelos botões.

10.2 Contribuição técnica do protótipo

O principal resultado não é a simples troca de imagens no monitor, mas a demonstração de uma cadeia completa de sistema embarcado: comandos possuem formato e integridade; o hardware decide se pode executá-los; um datapath aritmético processa dados em ponto fixo; uma memória guarda resultados; e o estado é exposto em uma interface HDMI. A implementação mostra, em pequena escala, a mesma separação encontrada em sistemas reais entre software de alto nível, protocolo, controle digital, acelerador dedicado e camada de observabilidade.

10.3 Limitações e decisões de engenharia

A principal limitação é a ausência de uma interface elétrica em tempo real entre uma CPU ARM64 física e a FPGA. O QEMU executa o binário AArch64, porém não representa um fio UART conectado à Tang Nano. Essa limitação foi tratada explicitamente por co-simulação, enquanto o hardware físico utiliza botões como fonte local de comandos. Outra decisão importante foi abandonar um framebuffer completo: a memória interna disponível na Tang

Nano 4K não é suficiente para armazenar 640 x 480 pixels a um byte por pixel, portanto os padrões finais são calculados proceduralmente e a BSRAM é reservada para telemetria.

10.4 Rastreabilidade das evidências

As evidências foram organizadas em cinco grupos: ARM64, simulação, waveforms, Gowin e hardware. Essa organização permite relacionar cada afirmação do relatório a um artefato observável: terminal para build e testes, GTKWave para comportamento temporal, Gowin para implementação física e fotos do monitor para o comportamento final da placa.

11 CONCLUSÃO

O projeto final consolidou com sucesso os conceitos introduzidos ao longo dos cinco TPs em uma arquitetura única e demonstrável. O lado ARM64 executado no QEMU gera oito pacotes estruturados e corretamente protegidos por CRC-8. A co-simulação recebe esses bytes por uma UART simulada e comprova o funcionamento do parser, do executor e das respostas, incluindo rejeição de CRC inválido e sequência duplicada. No lado FPGA, a Tang Nano 4K implementa entrada por botões, unidade MAC, BSRAM, controle de status e saída HDMI.

A implementação física foi sintetizada e roteada no dispositivo correto, utilizou os recursos dedicados esperados e apresentou timing compatível com o domínio de pixels de 25,2 MHz. A programação em SRAM foi concluída com sucesso e o monitor exibiu fisicamente os quatro modos previstos, incluindo a mudança do modo 11 após a geração de amostras de telemetria. Dessa forma, os resultados confirmam não apenas a produção de vídeo, mas a integração de controle, processamento, memória e observabilidade em hardware reconfigurável.

A limitação do enlace ARM64-FPGA físico permanece explicitamente documentada. Em vez de ocultá-la, o projeto adota uma estratégia verificável: integração de protocolo em co-simulação e validação do datapath no hardware real. Como evolução futura, um adaptador USB-UART TTL de 3,3 V ou uma plataforma ARM física poderia conectar os dois domínios eletricamente, permitindo medir latência e throughput do enlace em tempo real sem alterar a organização principal do sistema.